

计算机图形学

课程作业 1 光栅化器

[姓名待填写-学号待填写]

一、概述

本作业在教师提供的 C++ 渲染框架中完成单色三角形、规则网格超采样、二维几何变换、质心颜色插值、像素纹理采样和 MipMap 层级采样。六项功能共用三角形覆盖判定和采样缓冲：SVG 中的局部顶点先转换到屏幕坐标，覆盖的子样本再按图元类型写入颜色，最后统一平均到 8-bit RGB 帧缓冲。

主要实现位于 rasterizer.cpp、transforms.cpp 和 texture.cpp。纹理像素采样支持最近邻与双线性，层级采样支持第 0 层、最近层和相邻层线性混合；P 与 L 可独立组合。L_LINEAR 是本次任务 6 的必做内容，其中与 P_LINEAR 的组合构成三线性过滤。

实现还修复了起始代码中采样率切换后缓冲大小不匹配、零长度线段处理及最粗 MipMap 层未生成的问题。Windows 工程提供 CMake 编译入口与 VSCode 配置，提交目录同时保留修改源码、可执行程序、报告、机器人 SVG、自备纹理 PNG 和对比图。

实验图由本机 OpenGL 4.6 隐藏窗口自动调用真实 P、L、=、Z、S 键盘回调得到，S 回调读取显示缓冲并保存 PNG。窗口分辨率统一为 800×800，不将自动采集写成人工操作经历。每像素样本数记为 spp：1、4、16 spp 分别对应 1×1、2×2、4×4 网格。

二、绘制三角形

三条边表示为 $E_i(x,y)=a_ix+b_iy+c_i$ 。按有向面积统一方向，三个边函数都非负时计入覆盖，因而顺、逆时针顶点都正确。只遍历与屏幕相交的包围盒，1 spp 采样像素中心 $(x+0.5,y+0.5)$ 。

边界点包含在内，并允许浮点精度量级的极小容差，避免共享边白缝；退化三角形和非有限坐标直接跳过。

TriangleSetup 预计算系数和面积倒数，子采样行用 $E_i(x+\Delta x,y)=E_i(x,y)+a_i\Delta x$ 增量推进，减少重复算术。三类三角形共用覆盖遍历器，仅着色回调不同。没有基础实现对照计时，故不报告加速倍数。

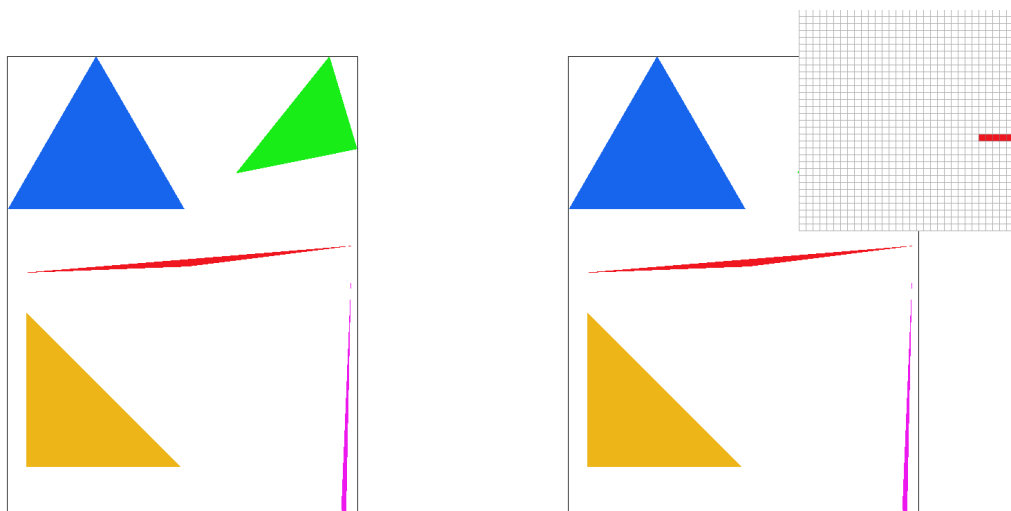


图 2.1 basic/test4.svg，800×800，1 spp，默认视角。右图启用像素检查器，光标固定 (166,378)；整图确认填充，放大区域展示细三角形的阶梯边缘。

三、基于超采样的反走样算法

令 $S = \text{sample_rate}$ 、 $n = \sqrt{S}$ 。每个像素的 $n \times n$ 子样本均匀分布在小格中心，第 (sx, sy) 个样本的位置为 $x + (sx + 0.5)/n$ 、 $y + (sy + 0.5)/n$ 。规则网格是本次作业要求的超采样方法；不以非规则样本或仅覆盖率统计替代它。

`sample_buffer` 将每个像素的 S 个 `Color` 连续存储，像素按行优先排列，子样本也按行优先排列。因此索引为 $(y \times \text{width} + x) \times S + sy \times n + sx$ 。构造、`set_sample_rate` 和 `set_framebuffer_target` 都按 $\text{width} \times \text{height} \times S$ 分配；采样率必须是正的平方数，尺寸计算检查溢出。

每帧开始将帧缓冲和采样缓冲清为白色。三角形仅改写命中的子样本；点和线通过 `fill_pixel` 把对应像素的所有子样本写为同色，使原有轮廓在提高采样率后仍可见。所有图元完成后，`resolve_to_framebuffer` 对同一像素的 S 个颜色求均值，再夹到 $[0,1]$ 并四舍五入为 0–255。

与直接写最终像素相比，这一管线保留了图元之间的逐子样本覆盖关系。边缘像素的最终颜色由被覆盖样本比例决定。例如白底上的红三角形覆盖 16 个样本中的 8 个，平均得到红色与白色各半的颜色，而不是整像素突然在红白之间跳变。

缓冲空间为 $\text{width} \times \text{height} \times S \times \text{sizeof}(\text{Color})$ ，相对于每像素样本数 S 线性增长。若只存一个累计颜色，后续遮挡图元便无法知道哪些子样本已被覆盖；分块或瓦片缓冲可以节约峰值内存，但要重新组织绘制顺序与 `resolve`。

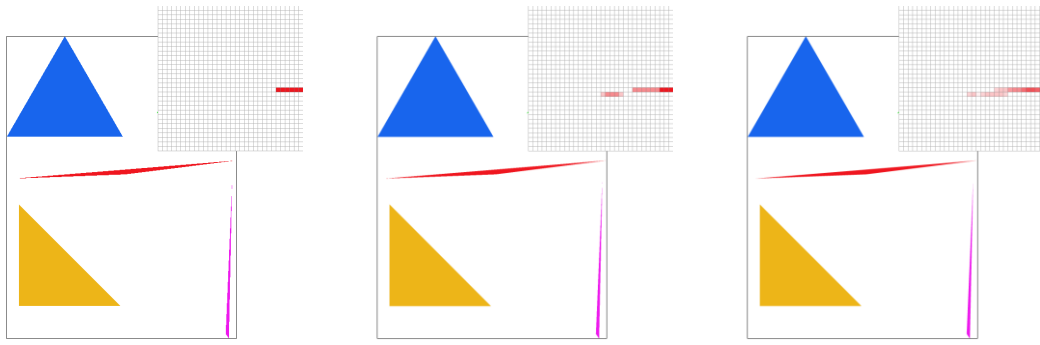


图 3.1 basic/test4.svg · 800×800 · 默认视角 · 1、4、16 spp 并排比较。三图都启用像素检查器，光标固定 (166,378)，分别使用 1×1、2×2、4×4 子样本网格。

1 spp 只有一个“命中或未命中”的判断，斜边形成明显台阶，细尖角也容易断续。4 spp 提供五档覆盖比例，边缘开始出现过渡；16 spp 提供十七档覆盖比例，细三角形的变化更连续。这里比较的是固定几何与固定视角下的采样密度，并非把不同缩放造成的视觉变化归于抗锯齿。

更高采样率同时提高覆盖测试、颜色插值、纹理过滤和 **resolve** 的工作量。规则网格仍是有限采样，对于比子格更细的高频结构可能留下走样；超采样改善误差，并不保证所有场景都完全没有锯齿。

四、几何变换

顶点使用齐次列向量 $(x,y,1)$ 。平移量位于矩阵第三列，缩放量位于主对角线；旋转角度乘 $\pi/180$ 后以 $\cos\theta$ 、 $\sin\theta$ 构造矩阵。 $M=T\times R\times S$ 依次缩放、旋转、平移。

SVG 父、子分组变换相乘，让部件绕自己的关节运动。绕 (cx,cy) 旋转为 $T(cx,cy)\times R\times T(-cx,-cy)$ 。原 robot.svg 核对基本矩阵，docs/my_robot.svg 保存可编辑的新动作。

新小人边跑边挥手：身体倾斜 -10° ，右上臂/前臂局部旋转 $-65^\circ/-35^\circ$ ；大腿为 $32^\circ/-30^\circ$ ，小腿再旋转 $-65^\circ/30^\circ$ 。头部再转 10° ，抵消身体倾斜。衣服蓝色，上臂青绿，头和手橙黄。

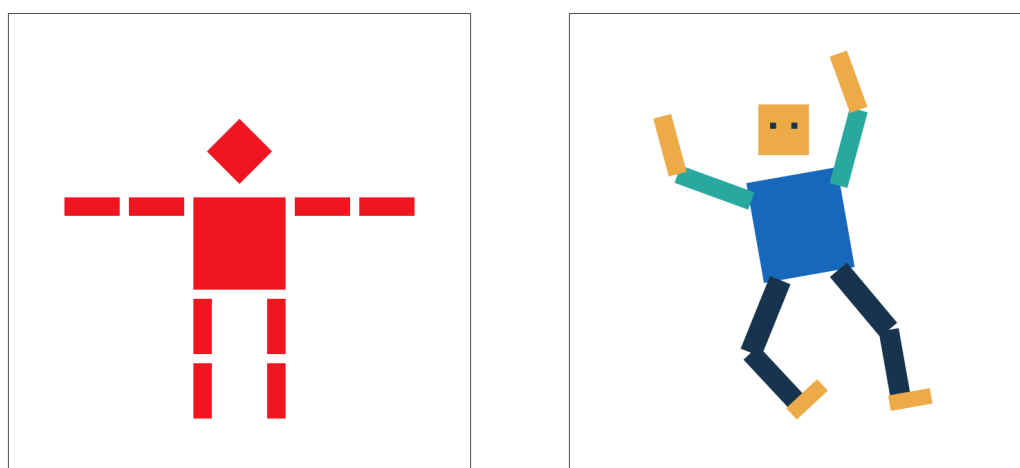


图 4.1 原 robot.svg 与 docs/my_robot.svg，800×800，各自默认视角，16 spp。两图具有相同输出分辨率和采样数，可对照部件关系与姿态。

五、质心坐标

对顶点 V_0 、 V_1 、 V_2 ，点 P 的质心坐标满足 $P = \alpha V_0 + \beta V_1 + \gamma V_2$ ，且 $\alpha + \beta + \gamma = 1$ 。三角形内部的三个权重非负；在某个顶点处，对应权重为 1，在它的对边处为 0。它们也可理解为对应子三角形的有向面积与总面积之比。

覆盖遍历器将三个对边边函数除以有向面积，得到 α 、 β ，并以 $\gamma = 1 - \alpha - \beta$ 计算第三个权重。颜色插值为 $C(P) = \alpha C_0 + \beta C_1 + \gamma C_2$ 。由于颜色按样本计算， 4×4 超采样时每个样本都得到自己的插值颜色，再在像素内平均。

如果三个顶点分别为红、绿、蓝，接近哪个顶点就得到更多对应分量，重心处三个分量相等，形成平滑渐变。`test7.svg` 将多个颜色三角形组成色轮，用于检验内部插值与共享边的连续性。

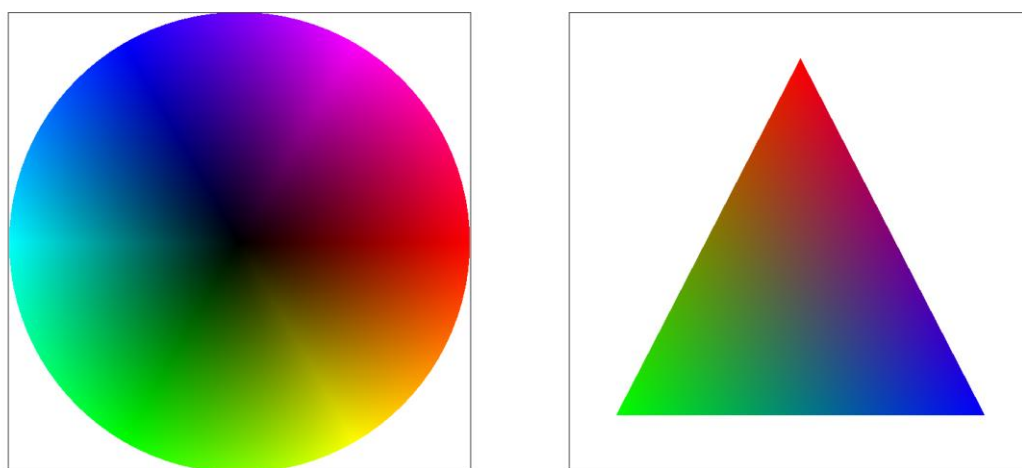


图 5.1 basic/test7.svg 与 docs/barycentric.svg， 800×800 ，默认视角，1 spp。红绿蓝顶点的颜色权重在线性插值中连续变化。

六、基于像素采样的纹理映射

屏幕样本以质心权重求 $uv(P)=\alpha uv_0+\beta uv_1+\gamma uv_2$ ，再由 `Texture::sample` 从离散纹素重建颜色。

UV 两端对应首末纹素中心， $x=u(W-1)$ 、 $y=v(H-1)$ ，越界值夹到边缘。最近邻将 x 、 y 四舍五入，读取一个纹素。双线性读取四邻纹素，用小数 tx 、 ty 先混合上下两行，再按 ty 混合两行；边界邻点也夹取。

例如 $top=(1-tx)C_{00}+txC_{10}$ ，最终 $C=(1-ty)top+ty\times bottom$ 。

MipLevel 以引用访问，避免复制图像。

本组选择 `texmap/test4.svg` 鹦鹉，`L_ZERO`、缩放 0.4，检查器置于眼部边缘 (348,230)。缺失的 PNG 从原框架固定提交恢复，地址与 SHA256 见 `dependency-assets.json`。



图 6.1 `texmap/test4.svg`， 800×800 ，缩放系数 0.4，检查器 (348,230)，`L_ZERO`、1 spp；左 `P_NEAREST`，右 `P_LINEAR`。



图 6.2 同一场景、视角、检查器与 800×800 分辨率，L_ZERO、16 spp；左 P_NEAREST，右 P_LINEAR。

在纹理被放大时，一枚纹素占据多个屏幕像素，最近邻的阶梯和块感最明显；双线性在纹素之间形成连续过渡，因而更适合本组放大区域。代价是突变边界和细小笔画会变柔和。“更平滑”不等于能够恢复原图中没有的细节。

将 1 spp 提高到 16 spp，可以平均同一屏幕像素中的多次 UV 取样，改善几何边缘和纹理边界。但如果十六个样本仍落在同一个纹素邻域，多次取样也不会增加源图细节。双线性控制纹理域的重建，超采样控制屏幕像素内的积分，两者处理的范围不同，可以组合使用。

纹理被大幅缩小时，一个屏幕像素覆盖许多纹素，最近邻只读取一个、双线性只混合四个，都可能遗漏高频信息并产生摩尔纹。该情形需要下一节的层级采样先使用经过低通过滤的纹理层。

七、MipMap 纹理映射

MipMap 将原图预先过滤为逐级降低分辨率的图像。层级采样根据屏幕像素在纹理上的足迹选择分辨率，使缩小后的像素取到更接近区域平均的颜色。它与某一层内的最近邻或双线性采样独立，因此可组合成六种模式。

在每个样本处记录 $uv(P)$ 、 $uv(P+(1,0))$ 、 $uv(P+(0,1))$ 。本实现利用质心函数对 x 、 y 的仿射梯度，预先求出两个 UV 差向量，与逐点重新计算这三个位置数学等价。偏移始终为一个屏幕像素，不能在高 spp 时改为一个子样本间隔，否则会低估纹理足迹。

用第 0 层宽 W 、高 H 分别缩放 UV 分量，得到 $dx=(W\Delta u_x, H\Delta v_x)$ 、 $dy=(W\Delta u_y, H\Delta v_y)$ 。足迹 $\rho=\max(||dx||, ||dy||)$ ，理想层级 $D=\log_2\rho$ ；放大或常量 UV 时使用 0 层，结果夹在 $[0, \text{最大可用层}]$ 内。尺寸使用 W 、 H ，而不是某个已经缩小的层。

L_ZERO 固定采样第 0 层； $L_NEAREST$ 选择 $\text{floor}(D+0.5)$ ；

L_LINEAR 分别采样 $\text{floor}(D)$ 和下一层，再按 D 的小数部分混合。与 P_LINEAR 组合时，每层内部双线性、层间再线性混合，构成三线性过滤。这属于任务 6 的完整实现。

起始 `generate_mips` 分配了最粗层却没有填充它。本次修复生成循环上界，使全部已分配层都实际过滤得到纹素；否则缩到最高 LOD 时可能错误地读到黑色。对仅一行或一列、奇数尺寸纹理也保留原框架的降采样规则。

自备 docs/sampling_texture.png 为 1024×1024 的程序生成图，包含 2、4、8 像素尺度的棋盘及每隔 128 像素的双色线。

docs/texture_demo.svg 将它映射到 24×24 网格构成的非均匀梯形，形成逐渐变化的纹理缩小程度。六图都使用默认视角、 800×800 、1 spp，并将检查器固定 (400,230)。

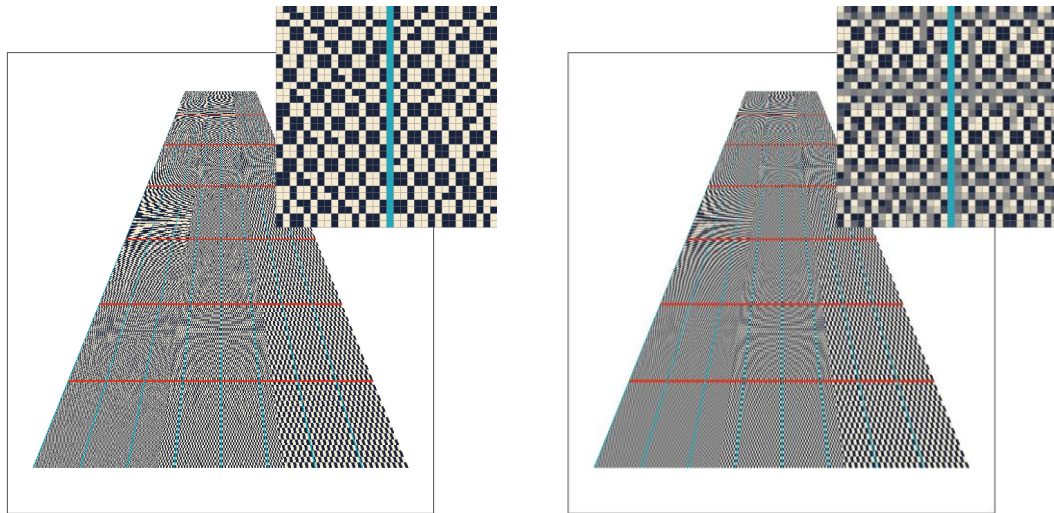


图 7.1 自备 PNG， 800×800 、1 spp，固定检查器 (400,230)；左 L_ZERO+P_NEAREST，右 L_ZERO+P_LINEAR。

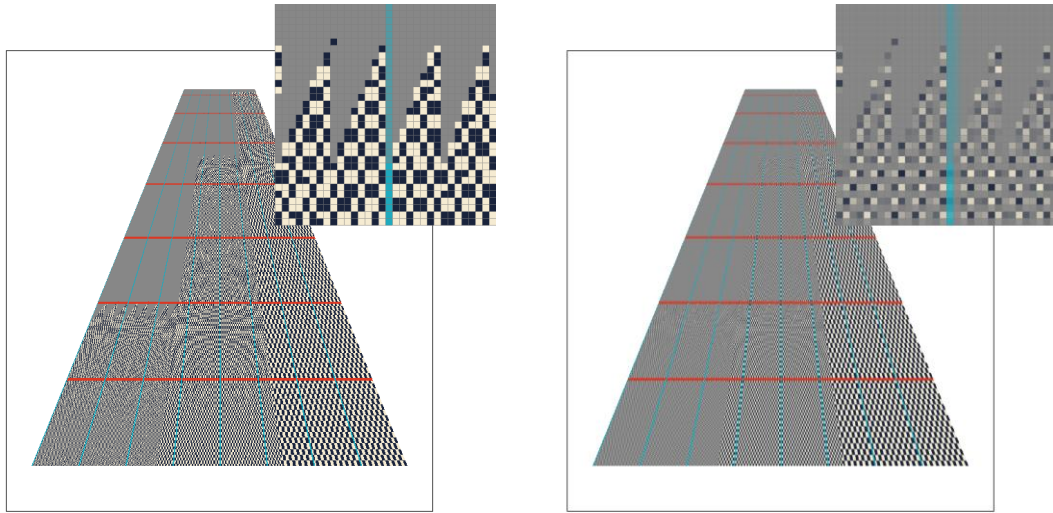


图 7.2 同一 PNG、SVG、视角、检查器和 1 spp；左 L_NEAREST+P_NEAREST，右 L_NEAREST+P_LINEAR。

固定 0 层的细密条纹容易在缩小区出现虚假的低频图案；最近层使用预过滤图像，能减弱这种混叠。P_LINEAR 进一步平滑层内纹素边界。四图保持其他条件一致，观察到的差异来自像素采样与层级策略。

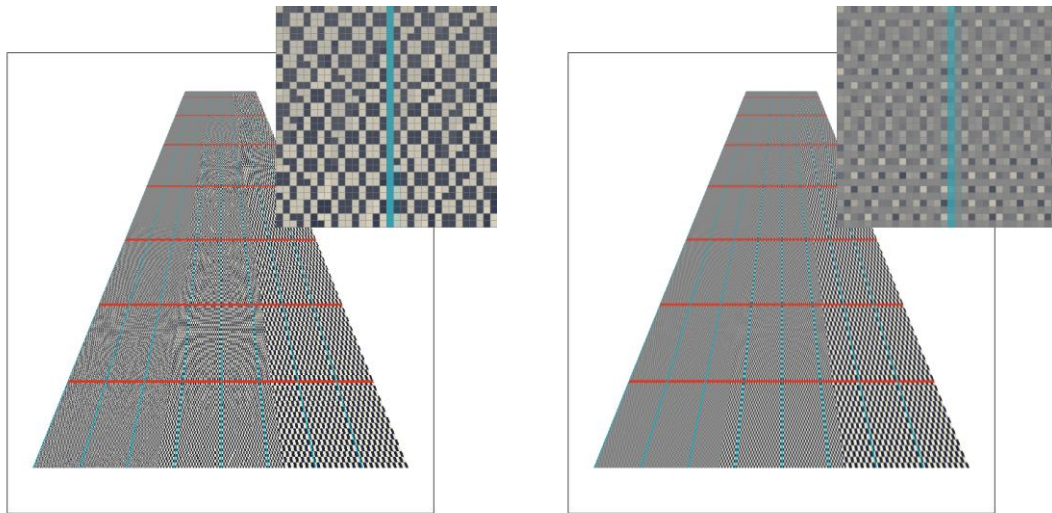


图 7.3 相同场景与参数；左 L_LINEAR+P_NEAREST，右

L_LINEAR+P_LINEAR。两张相邻层按连续 D 混合，避免整数层切换的突变。

最近邻每次读取 1 个纹素，双线性读取 4 个；最近层只使用一层，线性层级至多采样两层。以 P_LINEAR+L_LINEAR 为例，单个屏幕样本至多组合 8 个纹素，而 16 spp 还要对这一过程执行十六次。因此三种控制项可分别影响重建质量、缩小过滤和积分密度。

规则超采样缓冲随 spp 线性增长；改变 P 不增加采样缓冲，改变 L 的选择本身也不重新分配纹理。在本框架中纹理初始化即生成 MipMap，所以 L_ZERO 也保留已生成的层。对于长宽均逐级减半的常见纹理，完整金字塔像素总量约为原图的 $4/3$ ；该比例不适用于所有狭长纹理。

超采样主要改善屏幕积分，MipMap 主要抑制缩小纹理的高频混叠，双线性主要改善层内连续性。使用最近层仍可能随尺度改变出现层级跳变；三线性缓解层间变化，但各向同性足迹在斜视或高度拉伸区域可能模糊细节。各向异性过滤可作为后续改进，本次未将它列为已实现功能。

八、实践与思考

本次程序验证沿着覆盖判定、子样本缓冲、属性插值和纹理层级的数据流核对。首先检查顶点方向、边界、包围盒裁剪和退化三角形，再检查采样率与窗口变化时的缓冲长度；最后以小纹理的已知颜色和可计算的 UV 足迹核对像素与层级过滤，避免只凭一张图判断算法正确。

实际修复包括：统一三处缓冲分配与索引布局；让点和线写满该像素的子样本；对 8-bit 输出进行夹取和舍入；处理零长度线段、纹理边界和不合法参数；修复最后一级 MipMap 未填充。UV 梯度以一屏幕像素为单位，使改变超采样率时不会错误改变 LOD 公式。

OpenGL 实测还定位到教师 drawrend.cpp 的 glOrtho 近、远裁剪面同为 0，引发 GL_INVALID_VALUE。修复为 -1、1 后，使用窗口左上角对应的 glRasterPos2f(0,height)，移除原先的偏移，恢复有效贴屏投影。Windows 构建同时补充 WIN32 平台宏、GLEW_STATIC 及 texture.cpp 的 cstdint 引入。

算法验证记录为 verification/algorithm-tests.log，41 项全部通过、0 项失败。覆盖中心采样、顺逆时针、共享边无缝、裁剪、退化图元；4/9/16 spp 的精确半覆盖与点线广播；缓冲 resize/clear；可计算的质心权重；纹理角点、双线性均值、边界夹取；奇数与单维纹理、完整 1×1 最粗层；LOD 足迹、六组合及超采样下的一屏幕像素导数。实际截图参数、输出哈希和 OpenGL 信息留存在 verification/screenshots.json。

打包后的 bin/draw.exe 在 PATH 仅包含 Windows/System32 的环境下，对全部 32 个教师 SVG 成功生成 400×400 PNG，记录在 headless-tests.json；这也检查了运行资源与依赖。GDB 断点实际命中 rasterize_triangle 并记录调用链（gdb.log），验证 VSCode 所用调试器后端；本次未进行手动 F5 界面测试。

可视结果显示，单色多边形、机器人和色轮能够沿同一光栅化管线绘制；提高规则网格密度使细边缘更平滑；双线性改善纹理放大时的块感；层级采样改善缩小高频纹理的混叠。静态 PNG 能证明本次渲染结果，不能单独证明所有交互尺度下都没有闪烁，也不能代替算法边界测试。

Windows 复现先运行项目中的构建脚本，再将 SVG 文件或目录作为 draw.exe 参数。GUI 快捷键为 -= 调整 spp、P 切换像素采样、L 切换层级采样、Z 打开像素检查器、S 保存 PNG、空格复位视角；VSCode 配置用于选择编译与调试入口。报告所用的自动导出命令和参数另行留存在截图生成记录中。

后续可将采样缓冲按瓦片分配，降低高分辨率与高 spp 的峰值内存；也可在覆盖遍历中加入分块全内/全外测试，减少逐样本判断。改进时应固定测试场景和参数比较耗时与误差，不能把方法名称本身作为质量或性能提高的证据。